

ACM-ICPC TEAM REFERENCE DOCUMENT

Skill++ (Gogolev, Drachev)

Contents

1	General	1
1.1	C++ Template	1
1.2	Compilation	2
1.3	Check	2
1.4	How to solve problems	2
2	Math	2
2.1	Simpson	2
2.2	Prime	2
2.3	Modulo	2
3	Strings	2
4	Geometry	2
4.1	Manhatan Spanning	2

1 General

1.1 C++ Template

```
#pragma GCC optimize("O3,unroll-loops")
#define _CRT_SECURE_NO_WARNINGS
#define ln '\n'
#define All(x) (x).begin(), (x).end()
#define Allr(x) (x).rbegin(), (x).rend()
#define Allf(x) x.begin() + 1, x.end()
#define fi(a, b) for (auto i = (a); i <= (b); i++)
#define fj(a, b) for (auto j = (a); j <= (b); j++)
#define fo(a, b) for (auto o = (a); o <= (b); o++)
#define fq(a, b) for (auto q = (a); q <= (b); q++)
#define fe(a, b) for (auto e = (a); e <= (b); e++)
#define fw(a, b) for (auto w = (a); w <= (b); w++)
#define fdi(a, b) for (auto i = (a); i >= (b); i--)
#define fdj(a, b) for (auto j = (a); j >= (b); j--)
#define fdo(a, b) for (auto o = (a); o >= (b); o--)
#define fdq(a, b) for (auto q = (a); q >= (b); q--)
#define fde(a, b) for (auto e = (a); e >= (b); e--)
#define fdw(a, b) for (auto w = (a); w >= (b); w--)
#define fx(A) for (auto &x : (A))
#define fy(A) for (auto &y : (A))
#define fz(A) for (auto &z : (A))
#define pb push_back
#define mp make_pair
#define ins insert
#define sz(x) (int)x.size()
#include <string>
#include <vector>
#include <list>
#include <map>
#include <set>
#include <deque>
#include <queue>
#include <stack>
#include <bitset>
#include <algorithm>
#include <sstream>
#include <iostream>
#include <iomanip>
#include <cstdio>
#include <cstdlib>
#include <ctime>
#include <cstring>
#include <cmath>
#include <cstdlib>
#include <cassert>
#include <ctime>
#include <tuple>
```

```
#include <unordered_set>
#include <unordered_map>
#include <random>
#include <chrono>
#include <complex>
using namespace std;
#ifdef _DEBUG
#define LOCAL
#endif
mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
typedef long long ll;
typedef unsigned long long ull;
typedef long double ld;
typedef vector<bool> vb;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<ld> vld;
typedef vector<char> vc;
typedef vector<string> vs;
typedef vector<vb> vvb;
typedef vector<vi> vvi;
typedef vector<vll> vvll;
typedef vector<vc> vvc;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
template<typename A, typename B>
ostream& operator<<(ostream& os, pair<A, B> p) {
    os << "(" << p.first << ", " << p.second << ")";
    return os;
}
template<typename T>
ostream& operator<<(ostream& os, vector<T> v) {
    fi(0, sz(v) - 1) {
        os << v[i] << " ";
    }
    return os;
}
template<typename T>
ostream& operator<<(ostream& os, set<T> t) {
    for (auto z : t) {
        os << z << " ";
    }
    return os;
}
template<typename T1, typename T2>
ostream& operator<<(ostream& os, map<T1, T2> t) {
    cerr << endl;
    for (auto z : t) {
        os << "\t" << z.first << " -> " << z.second << endl;
    }
    return os;
}
#ifdef LOCAL
#define dbg(x) {cerr << __LINE__ << "\t" << #x << ": " << (x) << endl;}
#else
#define dbg(x) {}
#endif
#ifdef LOCAL
#define ass(x) if (!(x)) { cerr << __LINE__ << "\tassertion failed: " << #x << endl; abort(); }
#else
#define ass(x) assert(x)
#endif
void solve() {
}
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    cout << setprecision(20);
    #ifdef LOCAL
        freopen("input.txt", "r", stdin);
        freopen("output.txt", "w", stdout);
    #endif
    solve();
}
```

```
}
```

1.2 Compilation

```
#far association
g++ -std=c++17 -DLOCAL -O2 -Wall -Wl,--stack
    =67108864 -o !.exe
```

1.3 Check

```
:start
gen.exe
sol.exe
copy output.txt answer.txt
NAME.exe
fc output.txt answer.txt
if ERRORLEVEL 1 goto end
goto start
:end
```

1.4 How to solve problems

1. Math formulation
2. Using parameters
3. Go to theory
4. Way od solutions:
 - (a) fix parameter
 - (b) cases
 - (c) see on answer
 - (d) induction
 - (e) hypothesis
 - (f) finding of
 - (g) simplification
 - (h) reduction to other problem
 - (i) reduction to classic problem
 - (j) reduction of classic problem to our
 - (k) modeling
 - (l) generalization
 - (m) contrexamples
 - (n) full random
 - (o) invariant

2 Math

2.1 Simpson

```
double a = 0, b = 1;
const int n = 100000000;
double s = 0;
double h = (b - a) / n;
fi(0, n) {
    double x = a + h * i;
    s += fun(x) * ((i==0 || i==n) ? 1 : ((i&1)==0) ? 2 : 4);
}
s *= h / 3;
```

2.2 Prime

```
ll fun1(ll a, ll n) {
    ll t = n - 1;
    int q = 0;
    while(t % 2 == 0) {
        q++;
        t /= 2;
    }
}
```

```
ll v = pown(a, t, n);
fi(1, q) {
    ll w = mult(v, v, n);
    if(v != 1 && v != n - 1 && w == 1) return true;
    v = w;
}
if(v > 1) return true;
return false;
}

bool is_prime(ll n) {
    if(n < 2) return false;
    if(n == 2) return true;
    if(n % 2 == 0) return false;
    fi(1, 10) {
        ll a = next(1, n - 1);
        if(fun1(a, n)) return false;
    }
    return true;
}
```

2.3 Modulo

```
#define mod 1'000'000'007
int add(int a, int b){
    return (a + b) % mod;
}
int mult(int a, int b){
    return (ll)a * b % mod;
}
int pown(int a, ll n){
    if(n == 0) return 1;
    if(n % 2 == 0){
        int x = pown(a, n / 2);
        return mult(x, x);
    }
    return mult(a, pown(a, n - 1));
}
int obr(int a){
    return pown(a, mod - 2);
}
```

3 Strings

3.1 Suffix Array

```
struct Suff_arr{
    char LW = 3;
    int A = 256;
    string s0;
    string s1;
    vi arr;
    vi cost;
    vi rcost;
    vector<vector<pair<pair<int,int>,int>>> e1;
    vector<vector<pair<pair<int,int>,int>>> e2;
    vector<pair<pair<int,int>,int>> t;
    void Sort(vector<pair<pair<int,int>,int>>& t){
        fi(0,sz(t) - 1){
            e1[t[i].first.second].pb(t[i]);
        }
        fi(0,sz(e1) - 1){
            fx(e1[i]){
                e2[x.first.first].pb(x);
            }
            e1[i].clear();
        }
        t.clear();
        fi(0,sz(e2) - 1){
            fx(e2[i]){
                t.pb(x);
            }
            e2[i].clear();
        }
    }
}

void init(string s){
    s0 = s;
    s1 = s;
    s1 += LW;
    fi(0,sz(s0) - 1){
```

```

    if( (sz(s1) & (sz(s1) - 1)) == 0)break;
    s1 += s0[i];
}
vvi e(A + 1);
fi(0,sz(s1) - 1){
    e[s1[i]].pb(i);
}
fi(0,A){
    fx(e[i]){
        arr.pb(x);
    }
}
cost = vi(sz(arr));
rcost = vi(sz(arr));
fi(1,sz(arr) - 1){
    cost[i] = cost[i-1];
    if(s1[arr[i]] != s1[arr[i-1]]){
        cost[i]++;
    }
}
int SIZ = 2;
e1.resize(sz(arr));
e2.resize(sz(arr));
while(SIZ <= sz(arr)){
    fi(0,sz(arr) - 1){
        rcost[arr[i]] = cost[i];
    }
    t.clear();
    fi(0,sz(arr) - 1){
        t.pb( mp( mp(rcost[arr[i]], rcost[ (arr[i] + SIZ / 2)%sz(
            arr)]), arr[i]) );
    }
    Sort(t);
    cost[0] = 0;
    arr[0] = t[0].second;
    fi(1,sz(arr) - 1){
        arr[i] = t[i].second;
        cost[i] = cost[i-1];
        if(t[i].first != t[i-1].first)cost[i]++;
    }
    SIZ *= 2;
}

vi arr2;
fx(arr){
    if(x < sz(s0)){
        arr2.pb(x);
    }
}
arr = arr2;
}
vi LCP;
void init_lcp(){
    vi rarr(sz(arr));
    fi(0,sz(arr) - 1){
        rarr[arr[i]] = i;
    }
    LCP = vi(sz(arr));
    int len = 0;
    fi(0,sz(arr) - 1){
        if(rarr[i] == sz(arr) - 1){
            len = 0;
            continue;
        }
        len = max(0,len - 1);
        int x = i;
        int y = arr[rarr[i] + 1];
        while(x + len < sz(s0) && y + len < sz(s0) && s0[x + len]
            == s0[y + len]){
            len++;
        }
        LCP[rarr[i]] = len;
    }
}
}
};

```

3.2 Suffix Automaton

```

struct state {
    int len, link;
    map<char, int> next;
};
const int MAXLEN = 1'000'000;
state st[MAXLEN*2];

```

```

int siz, last;
void sa_init() {
    siz = last = 0;
    st[0].len = 0;
    st[0].link = -1;
    siz++;
}
void sa_extend (char c) {
    int cur = siz++;
    st[cur].len = st[last].len + 1;
    int p;
    for (p = last; p != -1 && !st[p].next.count(c); p = st[p].
        link) {
        st[p].next[c] = cur;
    }
    if (p == -1) {
        st[cur].link = 0;
    } else {
        int q = st[p].next[c];
        if (st[p].len + 1 == st[q].len) {
            st[cur].link = q;
        } else {
            int clone = siz++;
            st[clone].len = st[p].len + 1;
            st[clone].next = st[q].next;
            st[clone].link = st[q].link;
            for (; p != -1 && st[p].next[c] == q; p =
                st[p].link)
                st[p].next[c] = clone;
            st[q].link = st[cur].link = clone;
        }
    }
    last = cur;
}

```

3.3 Prefix function

```

vi prefix_function(string s) {
    int n = sz(s);
    vi pi(n);
    fi(1, n - 1) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) {
            j = pi[j - 1];
        }
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}

```

3.4 Z function

```

vi z_function (string s) {
    int n = sz(s);
    vi z (n);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) z[i] = min(r - i + 1, z[i - l]);
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}

```

3.5 Aho Korasik

```

struct Node {
    int leaf = -1;
    char c = '\0';
    Node* par = nullptr;
    map<char, Node*> nxt;
    Node* suf = nullptr;
    map<char, Node*> sufs;
    Node(char t, Node* p) {
        c = t;
        par = p;
    }
}

```

```

};
Node* root;
void add(string& s, int j) {
    Node* cur = root;
    fi(0, sz(s)- 1) {
        if(cur->nxt.count(s[i])) {
            cur = cur->nxt[s[i]];
        } else {
            Node* t = new Node(s[i],cur);
            cur->nxt[s[i]] = t;
            cur = t;
        }
    }
    cur->leaf = j;
}
Node* get_link(Node* a) {
    if(a->suf) return a->suf;
    if(a == root) {
        a->suf = root;
        return a->suf;
    }
    if(a->par == root) {
        return root;
    }
    char t = a->c;
    Node* cur = get_link(a->par);
    while(cur != root && !cur->nxt.count(t)) {
        cur = get_link(cur);
    }
    if(cur->nxt.count(t)) {
        cur = cur->nxt[t];
    }
    return a->suf = cur;
}
Node* go(Node* a, char c) {
    Node* cur = a;
    while(cur != root && !cur->nxt.count(c)) {
        cur = get_link(cur);
    }
    if(cur->nxt.count(c)) {
        cur = cur->nxt[c];
    }
    return cur;
}

string s;
ll n;
vector<string> a;
void init() {
    root = new Node('\0', nullptr);
    fi(1, n) {
        add(a[i], i);
    }
}

```

```

        for (auto &p : ps) { // rotate
            if (rot & 1) p.x *= -1;
            else swap(p.x, p.y);
        }
    }
    return edges;
}

```

4 Geometry

4.1 Manhattan Spanning

```

// Returns a list of edges in the format (weight, u, v).
// Passing this list to Kruskal algorithm will give the
// Manhattan MST.
vector<tuple<long long, int, int>> manhattan_mst_edges(
    vector<point> ps) {
    vector<int> ids(ps.size());
    iota(ids.begin(), ids.end(), 0);
    vector<tuple<long long, int, int>> edges;
    for (int rot = 0; rot < 4; rot++) { // for every rotation
        sort(ids.begin(), ids.end(), [&](int i, int j){
            return (ps[i].x + ps[i].y) < (ps[j].x + ps[j].y);
        });
        map<int, int, greater<int>> active; // (xs, id)
        for (auto i : ids) {
            for (auto it = active.lower_bound(ps[i].x); it !=
                active.end();
                active.erase(it++)) {
                int j = it->second;
                if (ps[i].x - ps[i].y > ps[j].x - ps[j].y) break;
                assert(ps[i].x >= ps[j].x && ps[i].y >= ps[j].y);
                edges.push_back({(ps[i].x - ps[j].x) + (ps[i].y - ps[
                    j].y), i, j});
            }
            active[ps[i].x] = i;
        }
    }
}

```